Progree

Your Career Our Mission

FRONTEND DEVELOPMENT INTERNSHIP

FINAL TASK SUBMISSION

Internship Tasks & Project Documentation

INTERNSHIP DETAILS

NAME	**Mahnoor Yasir**
STUDENT ID	**B4/1903**
ORGANIZATION	Progree
INTERNSHIP DOMAIN	Frontend Development
INTERNSHIP PERIOD	05 August 2026 – 05 September 2026
SUBMISSION	Tasks 1–4 Consolidated Final Report

COMPLETED TASKS

Progree

TASK 01

Professional LinkedIn Announcement

LinkedIn announcement & internship acceptance

TASK 02

Responsive SaaS Landing Page

LaunchFlow • responsive frontend experience

TASK 03

AERIS Weather Portal

Real-time weather application using REST APIs

TASK 04

Enterprise Analytical Administration Portal

Velox Analytics • reactive chart-based dashboard

A consolidated record of practical frontend development work completed during the internship.

Mahnoor Yasir • B4/1903 • Progree

Table of Contents

Introduction

Internship Context

This report documents the four tasks completed for the Progree Frontend Development Internship, a remote internship running from 05 August 2026 to 05 September 2026 under Student ID B4/1903. The internship offer was confirmed in writing on 4 August 2026 and announced publicly on LinkedIn as the first deliverable of the program.

Each task was assigned as a standalone frontend engineering exercise, building in scope from a single-page announcement (Task 1), through a responsive marketing interface (Task 2) and a live-data application (Task 3), to a multi-view analytics portal with data visualisation and full CRUD functionality (Task 4). No two tasks share a codebase; each is a separate project with its own repository folder, README and, where applicable, its own live demo.

Report Structure

Sections 1 through 4 document each task in turn, following a consistent structure: task overview and objective, official requirements, technologies used, implementation detail, screenshots of the working application, the project's GitHub repository, and a result summary. Section 5 draws the four tasks together into a short overall conclusion, Section 6 lists references, and Section 7 consolidates every GitHub and LinkedIn link in one table.

Throughout this report, technical claims are limited to what the underlying project's source code, README and documentation actually support. Where a source report distinguishes between behaviour that was independently tested and behaviour that was reviewed at the code level but not separately re-executed, that distinction is preserved rather than collapsed into a single claim of “working” or “passed.”

1. Task 1: Professional LinkedIn Announcement

1.1. Task Overview

Task 1 of the Progree Frontend Development Internship required a professional public announcement of the internship offer, posted on LinkedIn, along with supporting evidence of the offer itself. This section documents that announcement and the offer letter that confirmed the internship.

1.2. Objective

The objective of this task was to officially announce acceptance of the internship offer on a professional network (LinkedIn), in a way that reflects clear, professional communication and introduces the internship publicly.

1.3. Internship Information

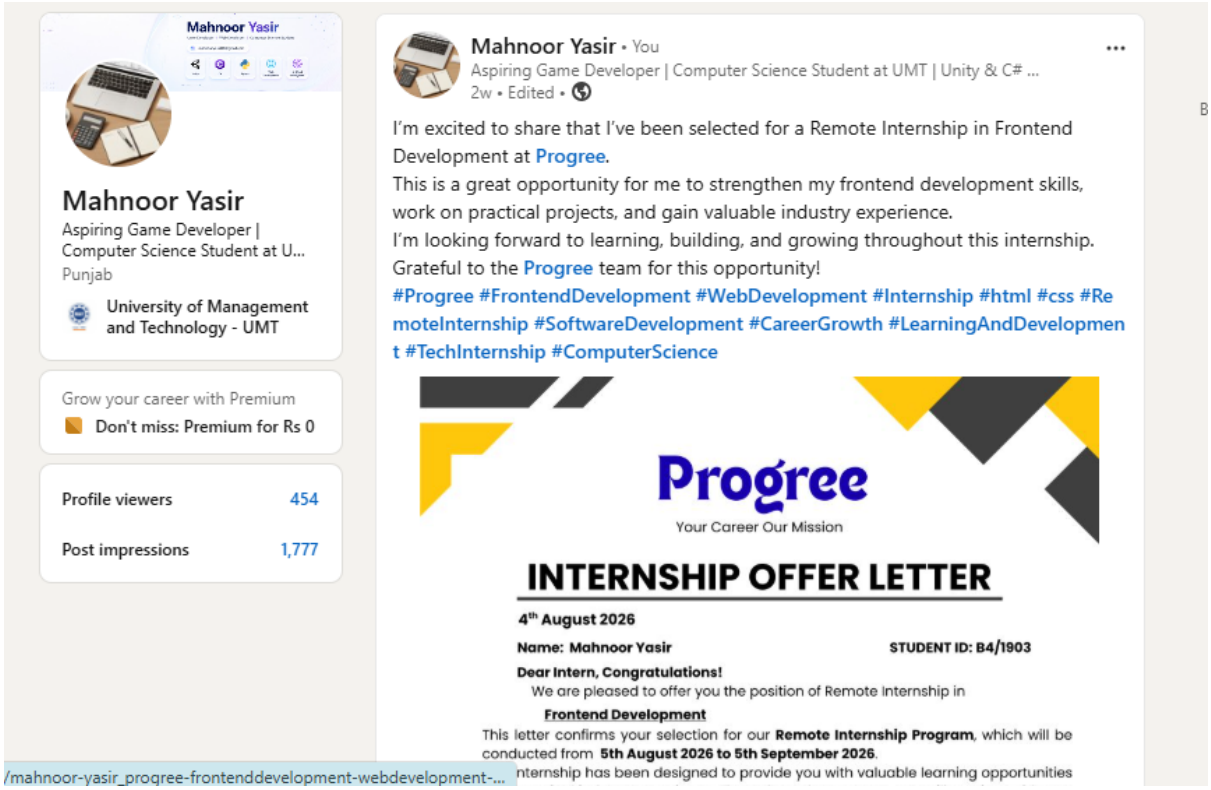
Field	Detail
Organization	Progree
Domain	Frontend Development
Internship Type	Remote Internship
Internship Period	05 August 2026 – 05 September 2026
Intern	Mahnoor Yasir
Student / Intern ID	B4/1903
Offer Letter Date	04 August 2026

1.4. LinkedIn Announcement

The announcement post read as follows:

> *“I’m excited to share that I’ve been selected for a Remote Internship in Frontend Development at Progree. This is a great opportunity for me to strengthen my frontend development skills, work on practical projects, and gain valuable industry experience. I’m looking forward to learning, building, and growing throughout this internship. Grateful to the Progree team for this opportunity!”*

1.5. LinkedIn Post and Offer Letter Screenshot



Progree

Your Career Our Mission

INTERNSHIP OFFER LETTER

4th August 2026

Name: Mahnoor Yasir **STUDENT ID: B4/1903**

Dear Intern, Congratulations!

We are pleased to offer you the position of Remote Internship in

Frontend Development

This letter confirms your selection for our **Remote Internship Program**, which will be conducted from **5th August 2026 to 5th September 2026**.

This internship has been designed to provide you with valuable learning opportunities and practical industry experience. Throughout the program, you will receive guidance, training, and mentorship to help you enhance your technical skills and gain a deeper understanding of development through hands-on projects and assignments.

As an intern, you are expected to:

- **Complete all assigned tasks and projects responsibly.**
- **Follow the instructions and guidelines provided by the Progree Team.**
- **Maintain professionalism and commitment throughout the internship period.**
- **Participate actively in learning activities and training sessions.**

We are excited to have you join our team and look forward to supporting your professional growth. We believe this internship will help you build the skills, knowledge, and experience needed for future career opportunities.

We wish you a rewarding, enjoyable, and successful internship experience with Progree.

Sincerely,

Umar Mushtaq
Founder & CEO

APPROVED Progree APPROVED

Phone: +92 314 1816693 **Email:** progree.edu@gmail.com **Address:** Rawalpindi , Pakistan

Figure 1.1: LinkedIn Internship Announcement, showing the attached Progree Internship Offer Letter (Intern ID B4/1903, period 05 August 2026 – 05 September 2026).

1.6. Official Internship Offer Letter

The offer letter, issued by Progree on 4 August 2026, confirms selection for the Remote Internship Program in Frontend Development, running from 5 August 2026 to 5 September 2026, under Student ID B4/1903. The letter is reproduced as an attachment to the LinkedIn post in Figure 1.1 above.

1.7. Project Links

LinkedIn Post: https://lnkd.in/p/dyQWVx8g

GitHub (Task 1 folder): https://github.com/mahnoor-yasir/Progree_Internship/tree/main/Task%201%20LinkedIn%20Announcement

1.8. Task Outcome

The internship offer was announced publicly and professionally on LinkedIn, with the official Progree offer letter attached as supporting evidence. This satisfied Task 1 of the internship and set the public record for the remaining three tasks.

2. Task 2: Semantic Mobile-Responsive Marketing Landing Page

2.1. Task Overview

Task 2 required a single-page product marketing interface, built with semantic HTML5 and custom CSS (Flexbox or Grid), and verified against explicit media queries across mobile and tablet boundaries with no horizontal scrolling at any width. LaunchFlow a fictional SaaS project-management product with the tagline “Plan better. Build faster. Launch smarter.” was built for this task.

2.2. Objective

Code a single-page product marketing interface with fully responsive layouts.

2.3. Official Requirements

- Write clean semantic HTML5 structures styled with custom CSS layout modules (Flexbox or Grid).
- Enforce responsive viewport grid changes across multiple mobile and tablet device boundaries using explicit CSS media queries, completely eliminating horizontal scrolling layout bugs.

2.4. Technologies Used

- HTML5 — semantic landmark structure (header, nav, main, section, article, footer)
- CSS3 — custom design system with CSS custom properties, Flexbox and Grid, no UI framework
- Vanilla JavaScript (ES6+) — five modules (ui.js, auth.js, pricing.js, dashboard.js, main.js)

LaunchFlow is hand-written throughout: one index.html (779 lines), three CSS files totalling 1,670 lines, and five JavaScript modules totalling 957 lines. There is no framework, no build step, and no backend.

2.5. Semantic HTML5 Structure

The page uses real landmark elements rather than generic divs: header, nav, main, section, article and footer describe what each part of the page is, which lets the CSS restyle content freely across breakpoints without the document losing meaning. This also gives screen readers an accurate outline and keeps the codebase maintainable.

2.6. Layout and Styling

The layout uses CSS Grid for the header, feature grid, bento panel, pricing cards and dashboard grid, and Flexbox for the navigation bar, button rows and footer. Every value is driven by CSS custom properties defined in style.css, so spacing, color and typography stay consistent across the site without repeated hard-coded values.

2.7. Responsive Design

The responsive strategy is mobile-first, implemented as six explicit breakpoint layers in responsive.css: ≤480px, 481–767px, 768–1023px, 1024–1279px, ≥1280px and ≥1600px. Each layer is written as a complete, self-contained override for its width range, which keeps the CSS cascade from resolving conflicts between unrelated rules.

2.8. Mobile and Tablet Adaptation

Below 1024px, the header navigation collapses from an inline bar into a slide-in drawer, sized with width: min(320px, 86vw) so it never itself overflows a narrow device. The feature grid, bento grid and pricing cards step from one column up to two, three or four columns as width increases, and every image and modal panel is constrained with max-width: 100% and fluid container widths rather than fixed pixel dimensions — the specific technique that prevents the horizontal-scroll bug the task named directly.

2.9. Key Features

- Marketing page: hero, feature grid, bento panel, product tour tabs, integrations grid, FAQ accordion, pricing grid, and a light/dark theme toggle
- Simulated signup and login backed by localStorage, with a personalised authenticated dashboard
- Three-tier pricing model with a monthly/yearly billing toggle
- Demo payment flow with real Luhn card-number validation, input formatting, and a masked payment method (no real payment is processed)

2.10. Interactive Product Features

Beyond the static marketing content, the page includes several small interactive systems: product tabs that let a visitor preview four product surfaces without leaving the page (cycled with ArrowLeft/ArrowRight and correct roving tabindex), a single-open FAQ accordion, an auto-advancing testimonial carousel that pauses on hover

or focus, animated stat counters that count up once a section scrolls into view, a scripted “LaunchFlow Intelligence” chat demo that appends canned responses, and non-blocking toast notifications for confirming actions.

2.11. Integrations

An integrations grid links to eight real external product websites (Slack, GitHub, Figma, Google Drive, Notion, Microsoft Teams, Zapier, Linear), each opening in a new tab with rel="noopener noreferrer" — the correct security practice for target="_blank" links regardless of whether the destination is trusted.

2.12. User Experience Design

The page repeats the same visual rhythm across every section (eyebrow label, heading, one-line supporting copy, then content), so a visitor learns the reading pattern within the first two sections. A single primary call-to-action colour is repeated at the top of the page, in the hero and at the closing section, while competing actions are styled as visually lighter ghost buttons. All eleven modals in the project share the same open/close mechanics, backdrop click-to-close, Escape-to-close and focus trap, and form errors appear inline next to the specific field rather than in a single top-of-form summary.

2.13. Accessibility

Accessibility was implemented as working behaviour rather than attribute decoration. Measures present in the markup and script logic include a visible skip-to-content link, ARIA roles and states on every custom widget (dialog/aria-modal on modals, tab/tabpanel on the product tabs, switch on the billing toggle), labelled form inputs with aria-live validation messages, full keyboard support with a proper focus trap inside open modals, focus restoration on modal close, and prefers-reduced-motion support that gives scroll animations and the carousel an instant fallback.

2.14. Performance

Performance comes largely from what was deliberately left out: there is no framework runtime to parse, no bundler output, and no external font or icon requests, since every icon is inline SVG and typography uses the operating system's own font stack. Event delegation is used for repeated interactive elements rather than one listener per element, and IntersectionObserver drives scroll-reveal, counters and scroll-spy instead of a scroll handler doing per-frame layout reads. Concatenating and minifying the separate CSS and JS files is listed as a recommended, not-yet-implemented optimisation for a production deployment.

2.15. Implementation Summary

Authentication and billing state are persisted through a small, consistently prefixed localStorage wrapper (LF.store), so every read and write goes through one place. Card validation is implemented as small, pure functions in pricing.js (formatCard, formatExp, luhn, brand), which keeps the payment form's event handlers thin wrappers around testable logic rather than validation buried inside DOM callbacks.

2.16. Screenshots

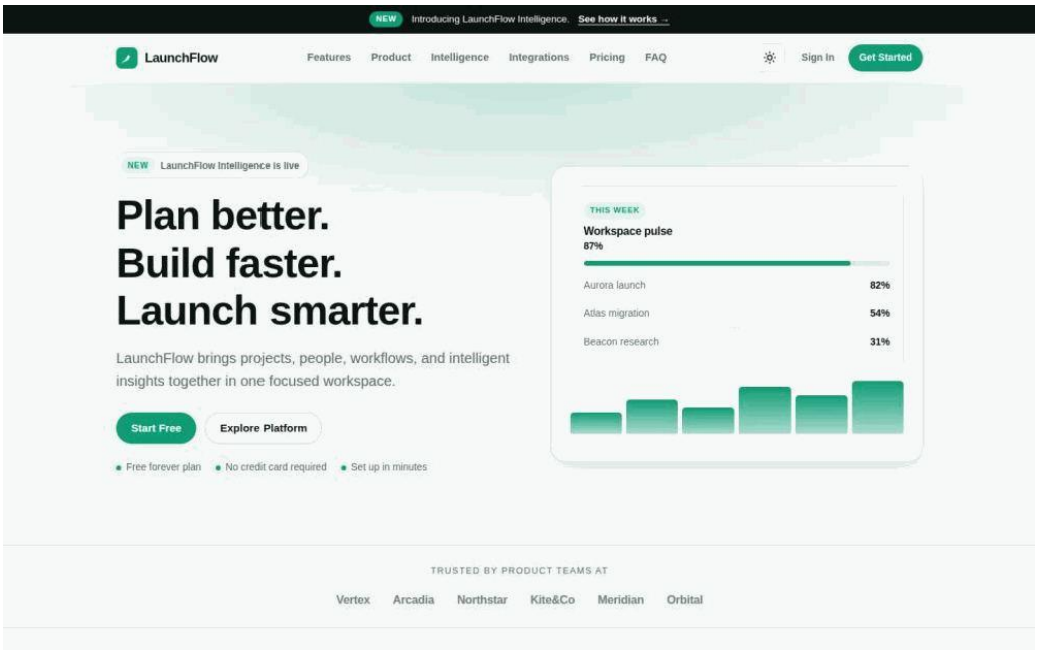


Figure 2.1: LaunchFlow Landing Page — Desktop View.

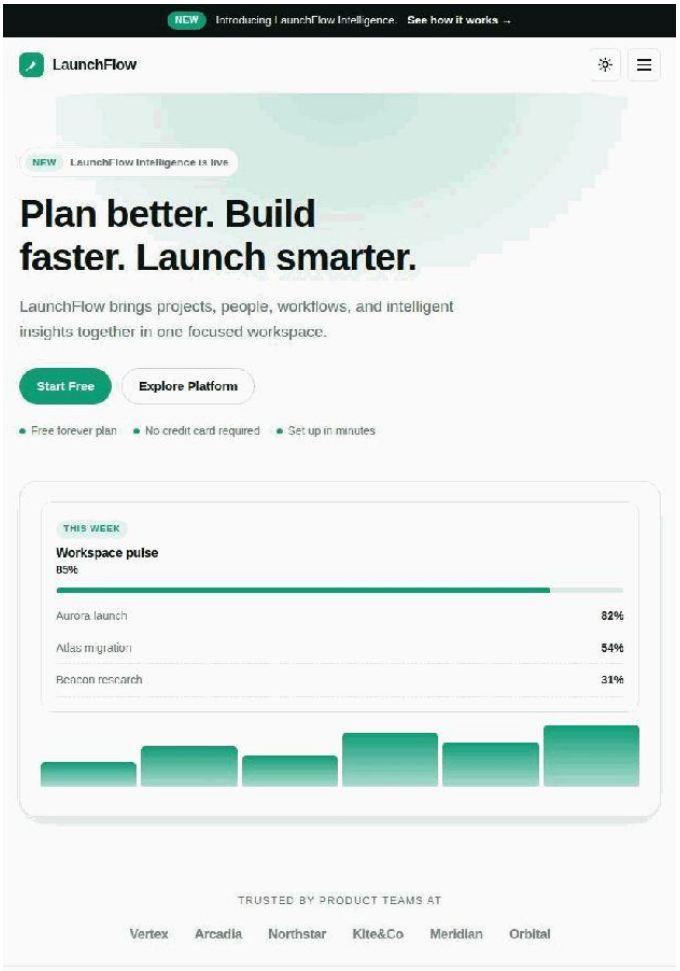


Figure 2.2: LaunchFlow Landing Page — Mobile Responsive View.

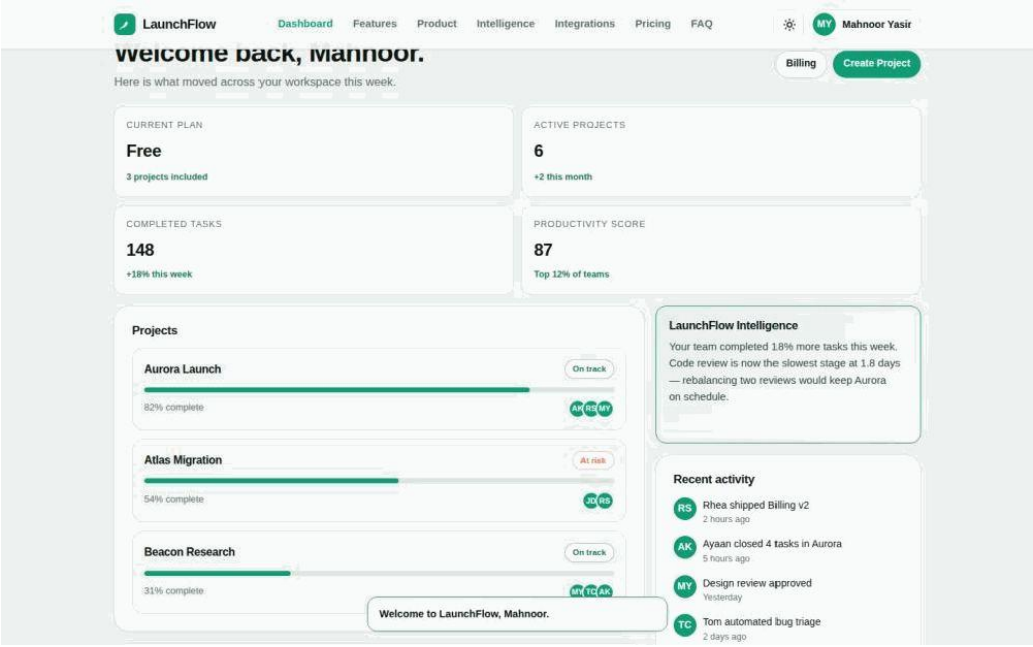


Figure 2.3: LaunchFlow Authenticated Dashboard ("Welcome back, Mahnoor").

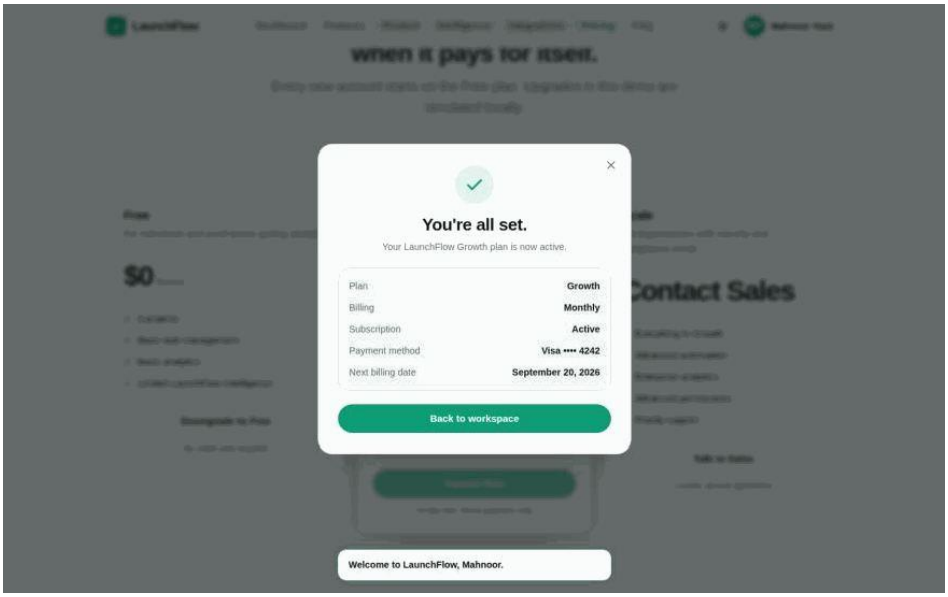


Figure 2.4: Demo Payment Flow — Luhn-Validated Card Success State.

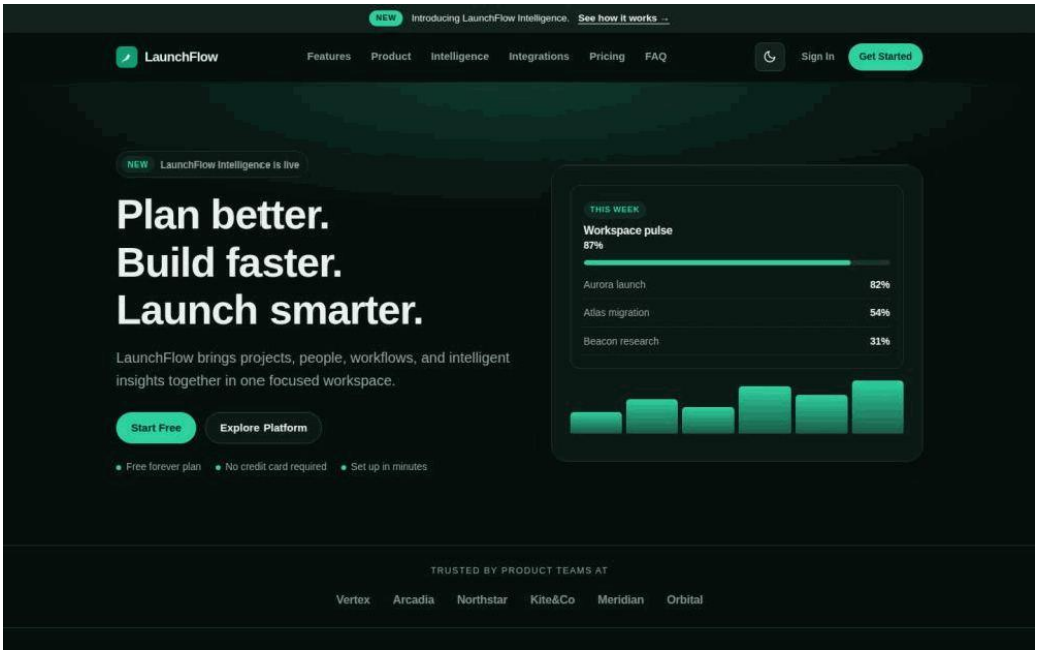


Figure 2.5: LaunchFlow Hero Section — Dark Theme.

2.17. Project Structure

The codebase is organised as a flat, framework-free structure so that every file's responsibility is easy to trace:

File / Folder	Responsibility
index.html	Full markup for the marketing page, dashboard and modals (779 lines)

File / Folder	Responsibility
css/style.css	Design tokens and base layout (575 lines)
css/components.css	Reusable UI components — cards, buttons, modals (943 lines)
css/responsive.css	Six breakpoint layers (152 lines)
js/ui.js	Shared UI helpers — toasts, modals, theme toggle (164 lines)
js/auth.js	Signup/login/session logic backed by localStorage (170 lines)
js/pricing.js	Billing toggle, upgrade flow, Luhn card validation (224 lines)
js/dashboard.js	Dashboard state, quick actions, activity feed (145 lines)
js/main.js	App bootstrap and event wiring (254 lines)

2.18. Testing

Testing combined the project's own documented verification matrix with an independent live re-verification carried out for this report: the project was served locally and driven through Chromium at three representative widths, and exercised through its real signup, pricing, upgrade and payment flows rather than only read as source code.

Flow	Result
Signup (valid data)	Account created, session started, redirected to dashboard with correct name
Signup (duplicate email)	Rejected with an inline alert; no duplicate account created

Flow	Result
Login (correct / incorrect credentials)	Correct credentials sign in; incorrect credentials show one generic inline error
Pricing toggle (monthly / yearly)	Growth price, upgrade summary and payment total all update together
Upgrade to payment (valid Luhn test card)	Review modal, then payment form, then success modal, in that order
Payment validation (invalid card / expired date / bad CVC)	Each rejected individually with its own inline error; submission blocked until resolved

The project's README documents responsive verification at 320, 360, 375, 390, 414, 480, 768, 820, 1024, 1280, 1440 and 1920px. For this report, three of those widths (375px mobile, 820px tablet, 1440px desktop) were independently re-rendered live in Chromium and are the source of every screenshot in this document; the remaining widths were reviewed against the breakpoint rules in responsive.css rather than individually screenshotted. The CSS relies on modern, broadly supported features (custom properties, Grid, Flexbox, clamp(), color-mix()); color-mix() is the newest of these and degrades to an unstyled but not broken fallback in older browsers, since it is only used for translucency effects rather than structural sizing.

2.19. Challenges and Solutions

Challenge	Solution
Coordinating six breakpoint layers without conflicting rules	Kept responsive.css strictly separate from component base styles, with each breakpoint block a complete, self-contained override
Keeping the mobile drawer usable at exactly 320px	Sized the drawer with width: min(320px, 86vw) so it is capped by the viewport rather than fighting it

Challenge	Solution
Persisting authentication state without a backend	A small, consistently prefixed localStorage wrapper (LF.store) so every read/write goes through one place
backdrop-filter on the header changing the mobile drawer's containing block	Identified as a spec-correct CSS Filter Effects behaviour; documented as a known finding with a listed fix (move #navMenu to be a direct child of body)
Client-side card validation without a payment SDK	Hand-written Luhn checksum, brand-prefix detection and input masking as small, pure functions in pricing.js

2.20. Learning Outcomes

- Writing six breakpoint layers made the difference between a page that looks fine at a couple of widths and one verified against a real, boundary-straddling width matrix.
- Keeping CSS custom properties as the single source of truth for spacing and colour made the dark theme and the responsive layers far easier to keep consistent than hard-coded values would have allowed.
- Reproducing the backdrop-filter containing-block behaviour firsthand was a clearer lesson in the CSS Filter Effects specification than reading about it would have been.
- Structuring card validation as small, pure functions (luhn, formatCard, brand) made the payment form's event handlers simple wrappers rather than logic buried inside DOM callbacks.

2.21. Project Links

GitHub Repository: https://github.com/mahnoor-yasir/Progree_Internship/tree/main/Task%202%20Responsive%20Landing%20Page

Linkdin Post: https://lnkd.in/p/dJvgbRE9

2.22. Result

The submitted project meets both official requirements: the markup is semantic throughout, the layout is custom-built with Grid and Flexbox with no UI framework, and six named breakpoint layers eliminate horizontal overflow through fluid containers and constrained media rather than overflow suppression. The responsive behaviour was independently re-verified at three representative widths (375px, 820px and 1440px) rendered live in Chromium, corroborated against the project's own twelve-point breakpoint matrix; this is not an automated cross-browser test suite, which the project lists as a future enhancement.

3. Task 3: Asynchronous REST API Weather Portal

3.1. Task Overview

Task 3 required a single-page interface that retrieves live data from a public REST API using asynchronous JavaScript, renders that data dynamically into the DOM, and handles loading and error states gracefully. AERIS — Intelligent Weather Portal, with the tagline “Weather intelligence, beautifully delivered,” was built for this task.

3.2. Objective

Build a data-driven application interface fetching and rendering remote data dynamically.

3.3. Official Requirements

- Write a single-page JavaScript application interface using React or Vanilla JavaScript.
- Use the Fetch API or Axios with async/await handlers to retrieve real-time weather information from a public endpoint such as the OpenWeatherMap API.
- Handle loading indicators and error states gracefully while dynamically injecting retrieved weather results into the user interface.

AERIS is implemented in Vanilla JavaScript ES6+ with the Fetch API, one of the two options the task explicitly allows, rather than React or Axios.

3.3.1. Requirement-to-Implementation Mapping

Official Requirement	AERIS Implementation
Single-page application	One index.html, no routing, no page reloads; state managed in-memory and in LocalStorage
Fetch remote weather data	api.js issues fetch() calls to OpenWeatherMap's geocoding, current-weather, forecast and air-pollution endpoints
async/await	Every network function is declared async and awaited from app.js's load()
Public weather API	OpenWeatherMap REST API (Geocoding, Current Weather, 5-day/3-hour Forecast, Air Pollution)

Official Requirement	AERIS Implementation
Dynamic rendering	ui.js writes every value into the DOM at runtime via renderAll(); nothing hard-coded in index.html
Loading states	UI.setLoading() shows a skeleton section and disables the search button; a duplicate-request guard prevents overlap
Error handling	A typed ApiError raised in api.js, mapped through a single lookup table in app.js
Responsive UI	css/responsive.css defines seven breakpoints (1440/1200/992/768/576/375/320px) on CSS Grid and Flexbox

3.4. Technologies Used

- HTML5 — semantic structure for every UI state and dashboard section
- CSS3 — design tokens, layout, theming and animation (style.css, animations.css, responsive.css)
- Vanilla JavaScript ES6+ — application logic across dedicated modules (api.js, app.js, ui.js, storage.js, config.js)
- OpenWeatherMap REST API — geocoding, current weather, 5-day/3-hour forecast and air-pollution endpoints
- Fetch API with async/await, wrapped in try/catch on every call
- AbortController — 12-second client-enforced request timeout
- LocalStorage — preferences, history, favourites and a response cache
- Geolocation API — “use my location” search entry point
- Inline SVG — a hand-built temperature-trend chart and sunrise/sunset arc, with no charting library

3.5. Application Interface

The interface is built with landmark elements rather than generic divs: a header containing the brand mark and controls, a main region holding every application state as a section, current-conditions and metric blocks marked up as article cards, and a footer. Every section carries an aria-labelledby pointing at its own heading,

and the search control is wrapped in a form with role="search" and a visually hidden label.

3.6. API Integration

api.js issues fetch() calls to OpenWeatherMap's geocoding, current-weather, forecast and air-pollution endpoints. No API key is exposed in this report; the key is read from local configuration at runtime.

3.7. Fetch API and Async/Await

Every network function (request(), geocodeCity(), fetchCurrentWeather(), fetchForecast(), fetchWeatherByCity()) is declared async and awaited from a single load() function in app.js. Current weather and forecast requests, which are independent of each other, are issued together with Promise.all rather than sequentially, for a faster first render and a single failure point to catch.

3.8. Weather Data Rendering

A dedicated ui.js module owns all DOM writes: the current-conditions card, six metric cards, a 24-hour outlook strip, the inline SVG temperature chart, a five-day forecast, a sunrise/sunset arc, and an optional air-quality panel. No data is hard-coded in index.html; everything is written to the DOM at runtime by renderAll(). A path-based defensive get() helper defaults missing fields to null, which is rendered as “Unavailable” rather than producing a broken UI or a raw JavaScript error.

3.9. Loading State

UI.setLoading() shows a skeleton/loading section and disables the search button while a request is outstanding. A duplicate-request guard (state.inFlight) prevents overlapping calls from fast double-submits or an auto-refresh tick.

3.10. Error Handling

api.js raises a typed ApiError with one of seven codes (NO_KEY, NOT_FOUND, NETWORK, TIMEOUT, AUTH, RATE_LIMIT, SERVER, UNKNOWN), which app.js maps to a specific, accurate title and message rather than a single generic error screen.

3.11. Key Features

- Unit and wind-unit switching without refetching (all math kept internally in Celsius/metric, converted only for display)
- Light/dark/system theming

- LocalStorage-backed favourites, recent searches, and a ten-minute response cache
- Optional conservative auto-refresh
- Geolocation-based “use my location” search
- Responsive layout tuned across seven breakpoints from 320px to 1440px+

3.12. Screenshots

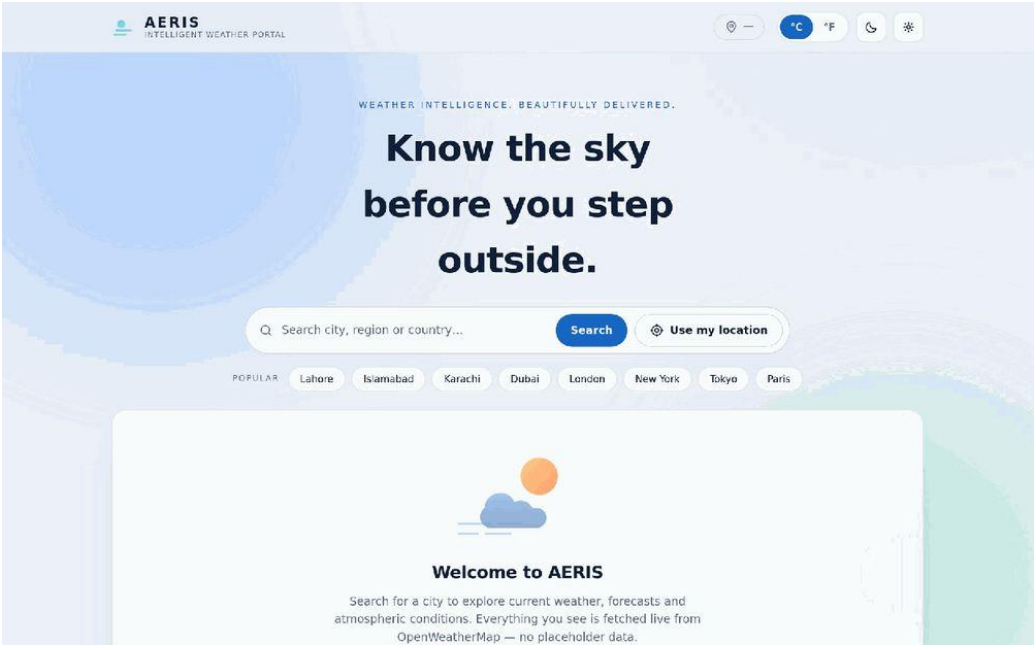


Figure 3.1: AERIS Weather Portal — Initial Interface.

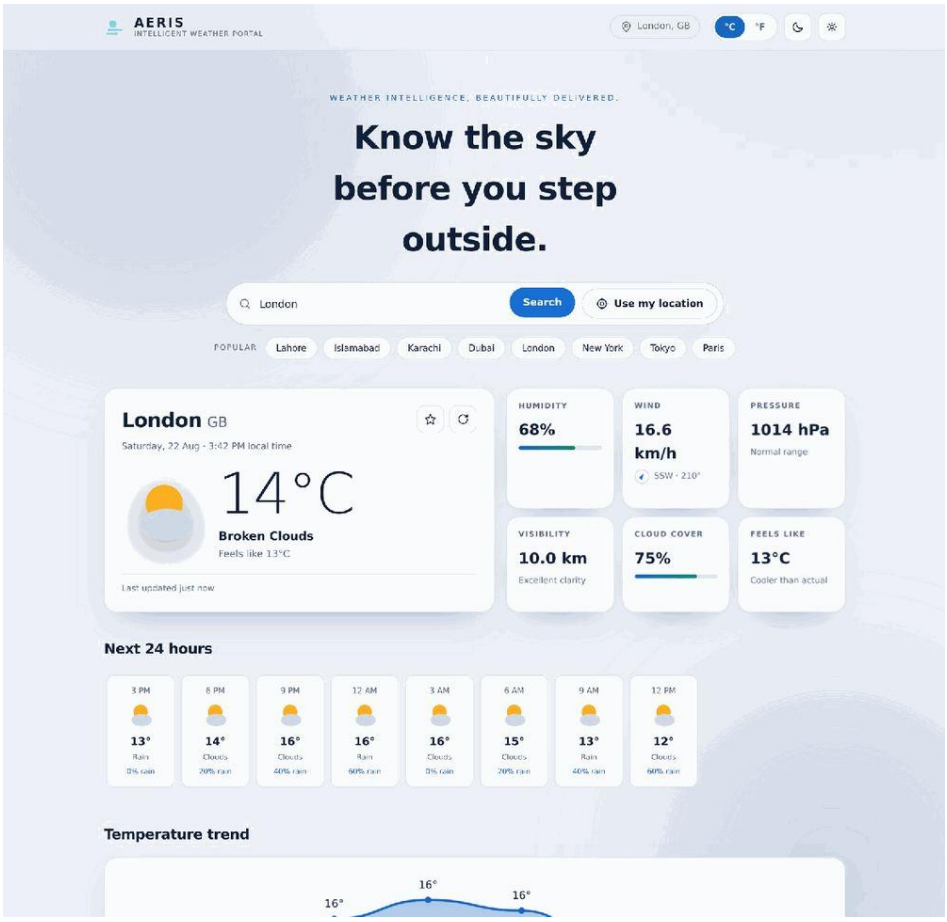


Figure 3.2: Weather Search Result — Current Conditions, Metrics and 24-Hour Outlook.

Figure 3.3: AERIS Weather Dashboard — Dark Theme.

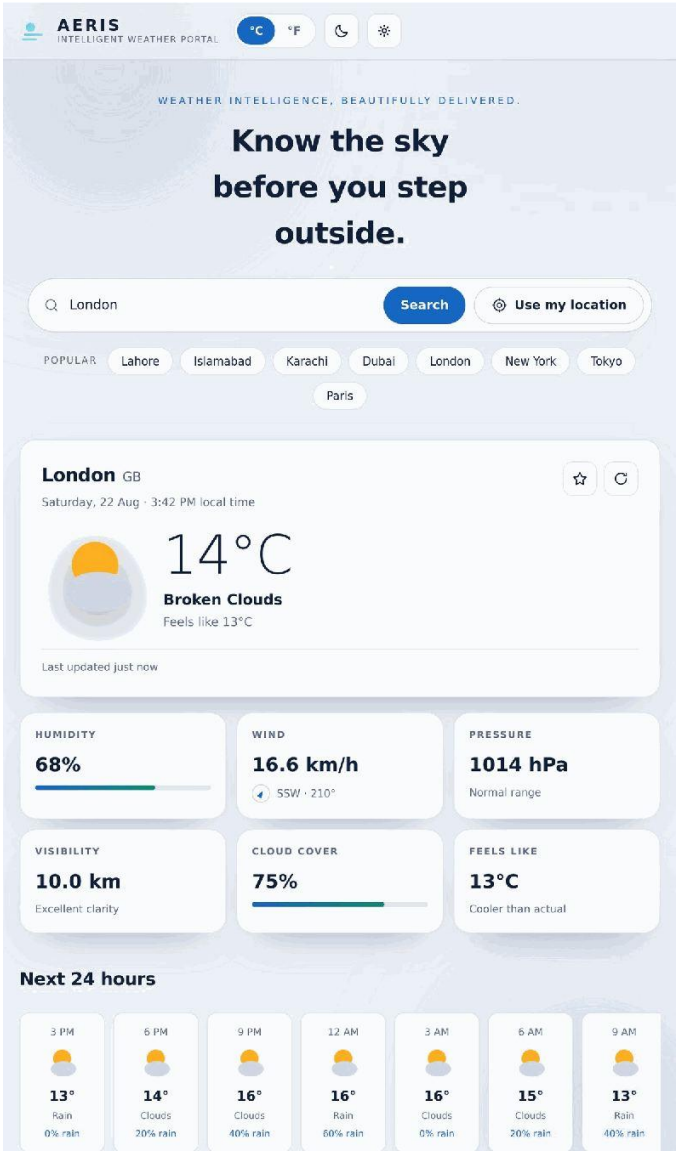


Figure 3.4: AERIS — Mobile Responsive View.

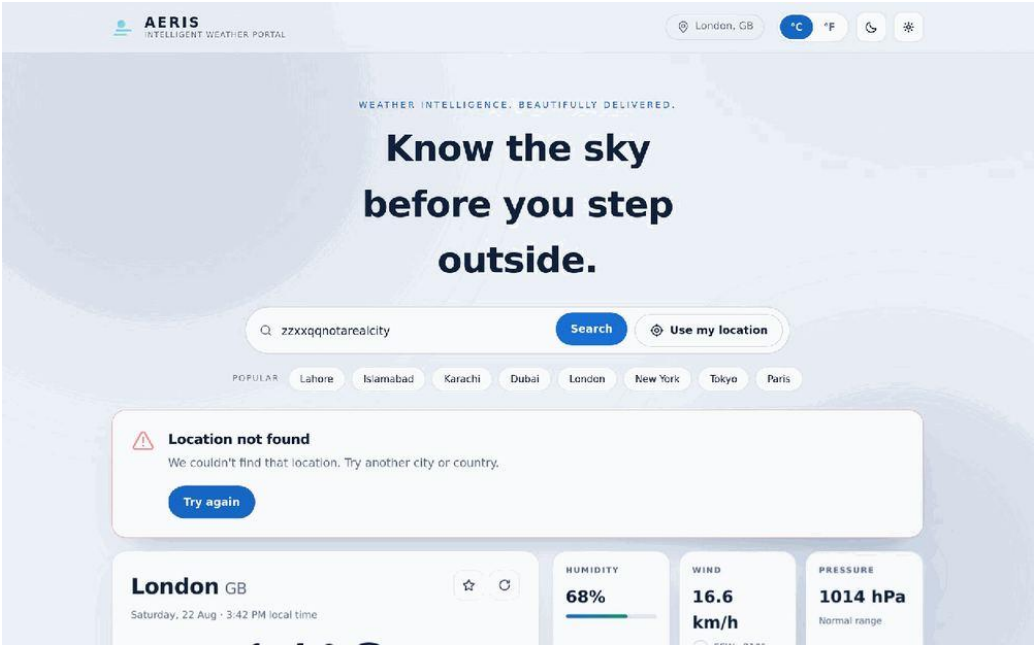


Figure 3.5: Error Handling — Location Not Found.

3.13. Project Architecture

The project follows a strict separation of concerns: api.js never touches the DOM, ui.js never performs network calls, and app.js orchestrates the two around a small shared state object. Every user action that needs data funnels through the same asynchronous load() function in app.js, which shows a loading state, awaits the network layer, and either commits a validated view-model to the UI or hands a typed ApiError to the error-message table — so a city search, a geolocation request, a favourite, a popular-city chip and a manual refresh all share the same loading and error handling rather than reimplementing it per feature.

- A cache lookup runs first; a fresh cached response (under 10 minutes old) renders instantly with no network call.
- On a cache miss, the location is resolved to coordinates through the OpenWeatherMap Geocoding endpoint.
- Current-weather and forecast requests run in parallel via Promise.all; air-quality is requested separately since it already resolves to null on failure.
- Raw JSON responses are validated and transformed into a defensive view-model before anything is rendered.
- Preferences, recent searches, favourites and the response cache are persisted to LocalStorage for the next visit.

3.14. Accessibility

Accessibility is addressed at the markup, ARIA and interaction level, though this is not a claim of formal WCAG certification, since no audit accompanies this submission. Concrete measures present in the source include semantic landmarks with a skip-to-content anchor, a visually hidden live status region announced on every meaningful state change, aria-pressed on toggle-style buttons, visible focus rings defined in CSS rather than suppressed, full keyboard operability of the search form, and prefers-reduced-motion support alongside a manual reduce-motion setting.

3.15. Weather-Driven Visual Themes

A skyFromIcon() helper in js/utils.js maps the OpenWeatherMap icon code and condition ID for the current location onto one of seven atmosphere keys (clear, clouds, rain, storm, snow, mist or night), with night detected directly from the “n” suffix OpenWeatherMap appends to night-time icon codes. The resulting key drives a background and accent treatment in the CSS, so the interface's mood shifts with the actual reported conditions rather than staying visually static regardless of whether it is showing a clear day or a storm.

3.16. Performance and Reliability

Performance and reliability come from a specific set of implemented measures rather than general claims: a 12-second request timeout via AbortController prevents an indefinitely spinning loading state, a duplicate-request guard prevents overlapping calls from rapid re-submits, a 10-minute response cache avoids repeat network calls for a location viewed again shortly after, current-weather and forecast requests run in parallel via Promise.all, and unit conversion happens entirely client-side so switching °C/°F never triggers a new fetch. Lazy-loading of below-the-fold sections and automated performance budgets are not present in the current codebase and are listed as future enhancements rather than claimed here.

3.17. Security Considerations

js/config.js contains an OpenWeatherMap API key used to authenticate requests during development and demonstration of this task. That key is not reproduced anywhere in this report, in any accompanying screenshot, or in any code excerpt. Because frontend JavaScript is publicly accessible, API credentials embedded in browser code should not be considered secret in a production environment; a production implementation should proxy OpenWeatherMap requests through a

backend layer and store the credential using environment variables or a server-side secrets manager instead of shipping it to the client. Before this repository is made fully public, the development key currently present in js/config.js should be rotated, since a public commit history would expose it permanently even if later removed from the latest revision.

3.18. Testing

No automated test suite (unit, integration or end-to-end) is present in the source. The tables below describe the project's recommended manual verification matrix rather than the results of tests actually executed and observed for this report, and are marked accordingly rather than reported as “Pass.”

Test Case	Expected Behaviour	Status
City search	Current-conditions card populates after a valid search	Recommended test
Geolocation search	“Use my location” resolves and loads local weather	Recommended test
Unit switching	°C/°F and wind-unit values update without a new request	Recommended test
Caching	Searching the same city twice within 10 minutes loads instantly the second time	Recommended test
Invalid city / empty search	NOT_FOUND message / validation message, no network call	Recommended test
Network failure / timeout	NETWORK / TIMEOUT typed error message shown	Recommended test

3.19. Challenges and Solutions

Challenge	Solution
Coordinating multiple independent async requests	Issued current weather and forecast together with Promise.all instead of sequential awaits

Challenge	Solution
Preventing duplicate in-flight requests	A state.inFlight guard checked at the top of load() before each new request
Handling a wide range of API failure modes	A typed ApiError with a code, mapped through a single ERRORS lookup table
Missing or partial response fields	A path-based defensive get() helper defaulting to null, rendered as “Unavailable”
Rebuilding a temperature chart without a library	A hand-computed linear scale plus a smoothed cubic-Bézier SVG path, redrawn per unit or location change
Grouping 3-hourly forecast entries into calendar days	Client-side grouping keyed by the location's own local calendar date, using its timezone offset

3.20. Learning Outcomes

- Structuring the async pipeline around a single, reusable request() function meant HTTP status mapping and timeout handling only had to be written and debugged once, then reused by five different endpoint functions.
- Promise.all only helps when requests are genuinely independent; fetchAirQuality() had to stay outside the main Promise.all since it already resolves to null on failure.
- AbortController's abort reason surfaces as a DOMException named “AbortError” from the rejected fetch promise, which meant checking err.name inside the catch block rather than assuming every rejection is a connectivity problem.
- Timezone-correct date rendering required working entirely in UTC-plus-offset, since a viewer in one timezone searching a city in another should see that city's own local time.
- Handling the Geolocation API's error codes separately, rather than as one generic failure, made the difference between “this feature is unavailable” and “please grant permission.”

3.21. Project Links

GitHub Repository: https://github.com/mahnoor-yasir/Progree_Internship/tree/main/Task%203%20Weather%20Portal

Linkdin Post: https://lnkd.in/p/dr_wadMg

3.22. Result

All eight items mapped against the Task 3 brief are met: a single-page application with no routing, integration with the OpenWeatherMap REST API, exclusive use of the Fetch API with async/await, fully dynamic DOM rendering, a skeleton loading state with a duplicate-request guard, typed error handling, and a responsive layout verified at seven breakpoints. Beyond the required scope, AERIS also adds unit switching, theming, favourites and history, a response cache, geolocation search, a hand-built SVG chart, and optional air-quality reporting.

4. Task 4: Enterprise Analytical Administration Portal

Velox Analytics Enterprise Intelligence Portal

4.1. Project Overview

Task 4 is a mini project requiring an interactive business analytics portal with rich, reactive data-visualisation components. Velox Analytics was built for this task: a frontend-only enterprise intelligence portal where mock JSON data is loaded once and then filtered, sorted, visualised and exported entirely in the browser, with localStorage used for persistence.

4.2. Objective

Architect an interactive business analytics data portal managing rich visual metric components.

4.3. Official Requirements

- Develop a multi-view client-side application using frontend routing frameworks.
- Integrate data visualization libraries such as Chart.js or Recharts to parse mock JSON profiles into responsive graphs.
- Build complete client-side searching, filtering, and multi-column sorting matrices.
- Build a dark-mode theme toggle state engine.
- Ensure full cross-browser compatibility and responsive behaviour.

4.4. Problem Statement

Business users evaluating sales performance need one interface to monitor KPIs, compare regions and categories, manage individual records, filter and sort large tables, generate shareable reports, and switch visual themes for different working conditions, all without depending on a live backend during prototyping. Velox Analytics implements this entire workflow on the client.

4.5. Technology Stack

Technology	Purpose
HTML5	Semantic structure for the login screen, view shell and page templates

Technology	Purpose
CSS3 (Grid, Flexbox, custom properties)	Layout of KPI grids, sidebars and tables; theme variables
JavaScript ES6+ (vanilla)	Routing, state, data and charts, with no framework dependency
Chart.js	Bar, line, doughnut and mixed charts for revenue, category and regional data
jsPDF	Generates downloadable PDF reports from the Reports view
Font Awesome	Icon set used across navigation, buttons and status indicators
Google Fonts (Inter)	Primary interface typeface
Browser localStorage	Theme/contrast/font-size, saved filters, notifications and dataset persistence

4.6. Application Architecture

The application is a single index.html shell that acts as the login screen and the host for every routed view. app.js contains a hash-based router that renders the Dashboard, Analytics, Reports, Data Manager, Settings, Help and Profile views into that shell without a page reload.

4.6.1. Project Structure

File / Folder	Responsibility
index.html	Login screen and application shell that routed views render into
assets/css/style.css	Global styling, theme variables, base layout, responsive breakpoints

File / Folder	Responsibility
assets/css/dashboard.css	Dashboard-specific layout (KPI grid, chart cards)
assets/css/components.css	Reusable UI components — buttons, modals, badges, toasts
assets/js/app.js	Router; renders Dashboard, Analytics, Reports, Settings, Help and Profile views
assets/js/dataService.js	Data store; CRUD operations; getStats/getTopCategory/getTopRegion; export
assets/js/filters.js	FilterEngine — sorting, category/status/region/date filters, saved filters
assets/js/theme.js	ThemeManager — dark/light theme, high contrast, font size, persistence
assets/js/notifications.js	Toast notifications and the notification centre
assets/js/charts/	Chart.js wrappers — chartConfig.js, barChart.js, lineChart.js, pieChart.js, mixedChart.js
assets/data/mockData.json	Mock dataset — sales, categories, regions, users, performance

4.7. Mock Authentication

The application opens on a login screen backed by a mock user directory rather than a real authentication server. Demo credentials are illustrative only and are not presented as production credentials. Signing in loads a role concept (Administrator, Manager, Analyst, Viewer) from the mock dataset, which is used to demonstrate role-aware UI elements rather than to enforce real access control.

4.8. Client-Side Routing

Navigation is handled entirely by a hash-based router in app.js, switching between views by matching the URL hash rather than making server requests, which keeps

the application a genuine single-page application with no routing framework dependency.

4.9. Data Architecture

assets/data/mockData.json holds the sales, category, region, user and performance data. dataService.js loads this once and exposes derived figures (getStats, getTopCategory, getTopRegion) from a single source, so the Dashboard, Analytics and Data Manager views stay consistent with each other after any create, update or delete operation.

4.10. Dashboard and Analytics

The Dashboard presents KPI cards and a Revenue Overview trend line, a Sales Distribution doughnut chart, and Regional Performance bars. The Analytics view extends this with date-range and category filters applied against the same underlying dataset, so figures on both views always agree.

4.11. Reactive Data Visualization

4.11.1. Chart.js Integration

Charts are implemented through dedicated modules — chartConfig.js, barChart.js, lineChart.js, pieChart.js and mixedChart.js — with Chart.js's responsive: true option combined with a CSS Grid layout for the chart containers. Chart.js does not automatically recolour an existing chart instance when the CSS theme variables change, so chart colour values are routed through chartConfig.js and chart instances are re-created on theme toggle to keep them in sync with the active theme.

4.11.1.1. Chart types implemented

- Bar chart — regional performance comparison
- Line chart — revenue trend over time
- Doughnut chart — sales distribution by category
- Mixed chart — combined revenue and order-count views in Analytics

4.12. Search, Filtering and Sorting

filters.js implements search against the sales dataset, plus category, status, region and date-range filters. Multiple filters are composed at the calling view rather than hard-coded as a fixed combination, so the Data Manager can apply search, category and status together. Multi-column sorting is implemented through a sortData(data, sortBy, sortDir) comparator invoked from column header clicks.

4.13. Pagination and Quick Filters

The Data Manager view paginates the sales table and offers quick filters (High Value, Recent, Completed, Pending) alongside localStorage-backed saved filters, so a frequently used filter combination does not need to be rebuilt on every visit.

4.13.1. Saved Filters and History

Filter combinations can be saved under a name and reapplied later, with the saved set persisted to the velox_saved_filters localStorage key rather than kept only in memory, so a user's preferred view of the data survives a page reload.

4.14. CRUD Operations

dataService.js implements full CRUD against the sales array, with main.js supplying the modal forms and validation that call it. addSale() appends a new record after the create modal validates required fields, then triggers a notification and re-renders the table. getSales(filters) and getSalesById(id) back both the Data Manager table and the record-detail view, and accept the same filter object produced by the search and filter controls. updateSale(id, updates) merges the supplied fields into the existing record, with the edit modal pre-filled from the current record so only changed fields are submitted. deleteSale(id) removes a single record and deleteSales(ids) removes a bulk selection made through the grid's row checkboxes, both gated behind a confirmation step in the UI. Every write path re-saves the updated array to localStorage, so changes persist across a browser refresh even without a backend database.

4.15. Reports and Export

The Reports view lists report entries such as Q1 Sales Summary, Team Performance Q1 and Financial Review March, each with a status badge and per-row PDF/CSV download actions. “Generate New Report” opens a modal offering a PDF or CSV download of the selected report.

4.15.1. PDF Export

downloadReport() constructs a jsPDF document and triggers a file download; this path is implemented for both the Reports list and the report-generation modal.

4.15.2. CSV Export

downloadReportCSV() and dataService.exportData('csv') both produce CSV output. The Reports UI labels one CSV action with an Excel icon, but the underlying implementation writes CSV rather than a native .xlsx workbook; this report describes that actual behaviour rather than the icon's implication.

4.16. Notification System

notifications.js implements addNotification() for four types (success, error, warning, info), each mapped to a Font Awesome icon and a default title. A transient toast gives immediate feedback for actions such as a CRUD write, while the notification centre keeps a running list with an unread-count badge (capped at “99+”) that is loaded from and persisted to localStorage, alongside clear-all and mark-all-read actions.

4.17. Theme and Accessibility

ThemeManager in theme.js persists theme, contrast and font-size preferences to localStorage under discrete keys (velox_theme, velox_contrast, velox_font_size), so each preference can be read and written independently. This provides the required dark-mode toggle, plus a high-contrast mode and a font-scaling control (80–120%) that were not explicitly required but strengthen the submission. Formal WCAG compliance is not claimed, as it was not independently tested.

4.18. Responsive Design

Layout responds at 1024px, 768px and 480px breakpoints defined in style.css and dashboard.css, collapsing the sidebar and moving to a single-column KPI and chart grid at narrower widths. Below these thresholds the sidebar collapses, KPI and chart grids drop from multi-column to single-column, and data tables and forms stack vertically, with the specific adjustments defined per breakpoint in the corresponding CSS file rather than through one global mobile stylesheet.

4.19. Cross-Browser Compatibility

The implementation relies only on standard DOM, CSS Grid/Flexbox and localStorage APIs, with no browser-specific or experimental features found in the source, which is a reasonable basis for compatibility across current versions of Chrome, Edge, Firefox and Safari. No formal cross-browser test log was included in the project, so this report distinguishes between implemented compatibility considerations (standards-based CSS/JS, no vendor-specific APIs) and recommended validation (an actual pass/fail run in each target browser, which should be performed and recorded before final submission).

4.20. Settings

The Settings page groups controls into three areas. Theme Preferences (dark mode, high contrast, font size) are functionally wired to ThemeManager and persisted to localStorage. The Notifications toggles and Account fields are present as UI controls

in settings.html, but no corresponding persistence or backend call was found for them in the reviewed JavaScript; they are treated here as UI-only pending confirmation, rather than claimed as fully functional.

4.21. Help Center

The Help Center includes six documentation cards, a 14-item FAQ accordion, a Quick Video Tutorials section, and a Keyboard Shortcuts reference (Ctrl+K search, Esc close modal, Ctrl+R refresh data, Ctrl+N new record).

4.22. UI/UX Design

The interface follows a conventional admin-dashboard pattern: a persistent sidebar for navigation, a top KPI row for at-a-glance metrics, chart cards below that, and a data table with inline, colour-coded status badges (Completed, Pending, In Progress, Failed). This summary-first, detail-second hierarchy keeps the dashboard scannable, and the same status-badge colour convention is reused across the dashboard's recent-activity table and the Data Manager grid.

4.23. Performance

All filtering, sorting and pagination happen client-side against an array already held in memory, so operations are effectively instant at the current mock-data scale. Chart configuration is centralised in chartConfig.js and reused across chart types rather than duplicated per chart, and the data grid is paginated so the DOM only renders the current page's rows rather than the full dataset. These are appropriate techniques at prototype scale; at production scale with a real backend, server-side filtering and pagination would be the next step.

4.24. Security Limitations

Because the project is a frontend-only prototype, several things that would matter in production are intentionally absent and are stated plainly rather than glossed over: authentication is a client-side credential check with visible demo credentials rather than a real login system, there is no server-side authorization or request validation, data lives in the browser's localStorage (per-browser, unencrypted, and not a substitute for a real database), and there is no audit trail beyond the in-app notification list. None of this is a defect in a frontend Task 4 deliverable; it is the expected shape of a client-only prototype, but it should not be described as production-secure.

4.25. Challenges and Solutions

Challenge	Solution
Keeping Dashboard, Analytics and Data Manager figures consistent after edits	Centralised all derived figures in dataService.js so every view reads the same source
Chart.js not re-colouring existing charts on theme change	Routed chart colours through chartConfig.js and re-created chart instances on theme toggle
Combining multiple simultaneous filters (search, category, status, region, date)	Composed filters at the calling view rather than hard-coding a fixed combination in FilterEngine
Persisting theme, contrast, font size and saved filters without a backend	Used discrete localStorage keys per concern rather than one shared blob

4.26. Requirement Compliance

Official Requirement	Implementation	Status
Multi-view client-side application	Hash-based router in app.js across seven views	Verified
Chart.js / Recharts integration	Chart.js with dedicated chart modules	Verified
Mock JSON profiles	assets/data/mockData.json loaded by dataService.js	Verified
Responsive graphs	Chart.js responsive:true plus CSS Grid chart containers	Verified
Client-side searching, filtering, sorting	filters.js search/filter/sort logic	Verified
Dark-mode theme toggle	ThemeManager in theme.js	Verified
Responsive design	Media queries at 1024/768/480px	Verified

Official Requirement	Implementation	Status
Cross-browser compatibility	Standard DOM/CSS APIs; no formal multi-browser test log	Not independently verified

Testing performed for this report was manual and exploratory against the running application. No automated test suite was included in the project, and items not directly exercised for this report are recorded honestly as “not independently verified” rather than assumed to pass.

4.27. Screenshots

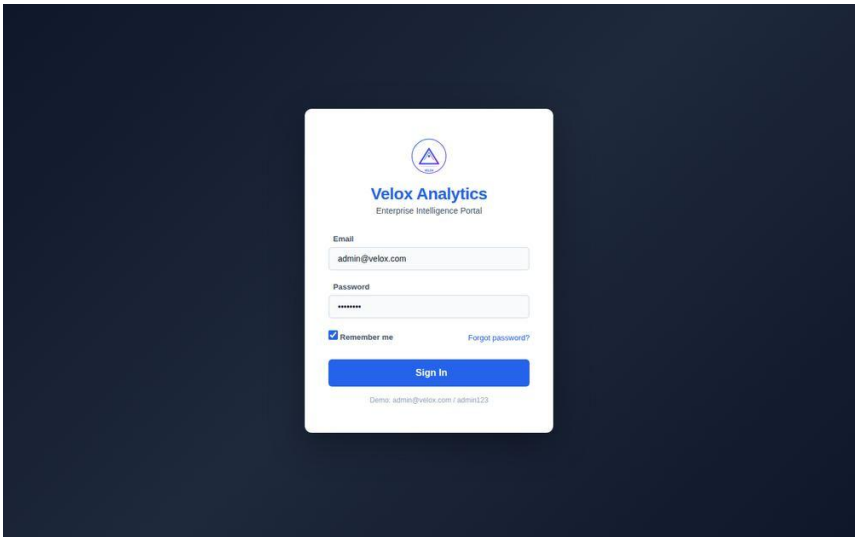


Figure 4.1: Velox Analytics — Login Screen.

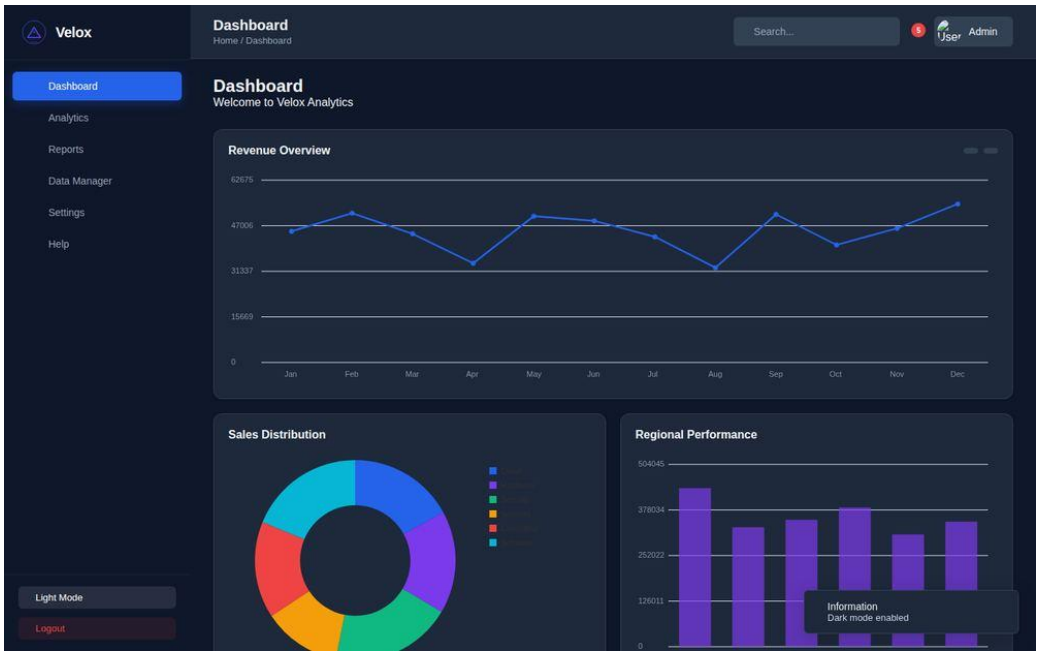


Figure 4.2: Enterprise Dashboard — Dark Theme (Revenue, Sales Distribution, Regional Performance).

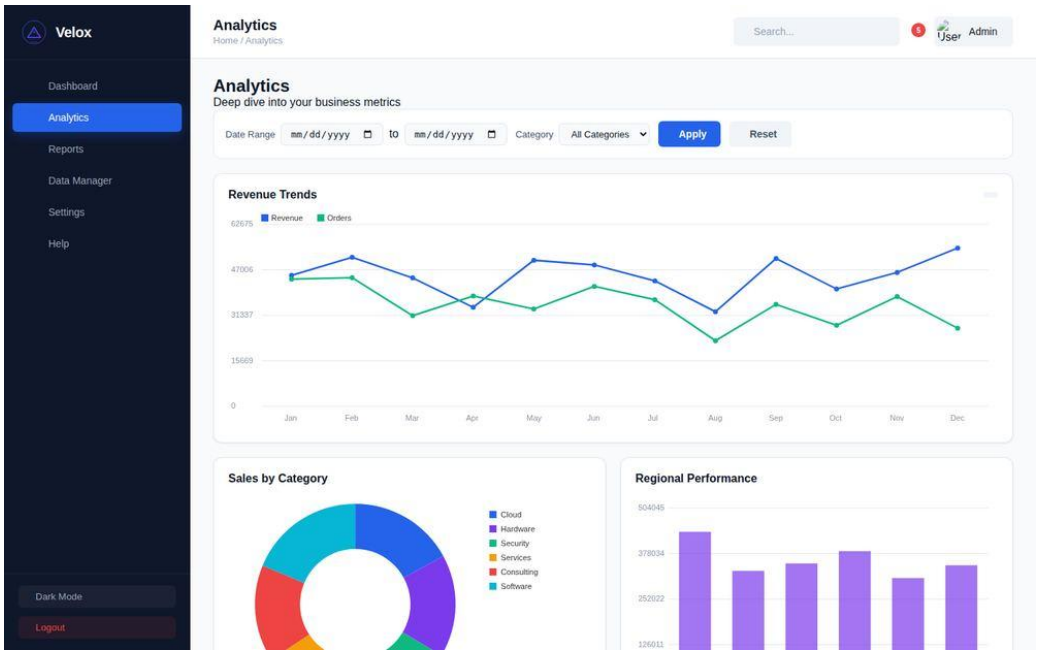


Figure 4.3: Analytics View — Reactive Charts with Date and Category Filters.

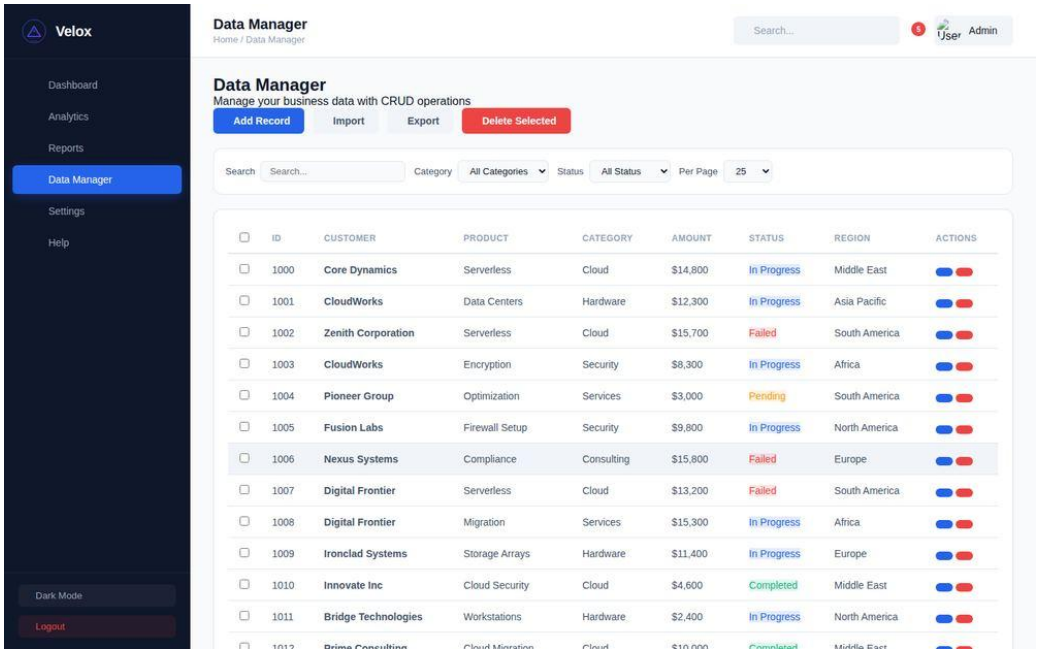


Figure 4.4: Data Manager — Searching, Filtering, Sorting and CRUD Operations.

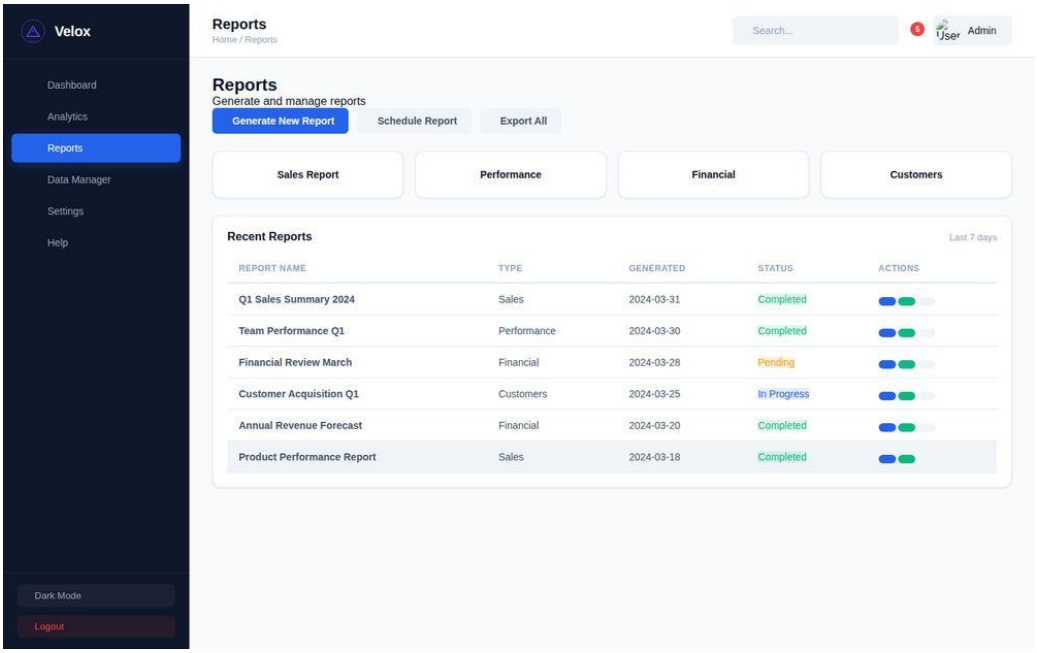


Figure 4.5: Reports View — PDF/CSV Export.

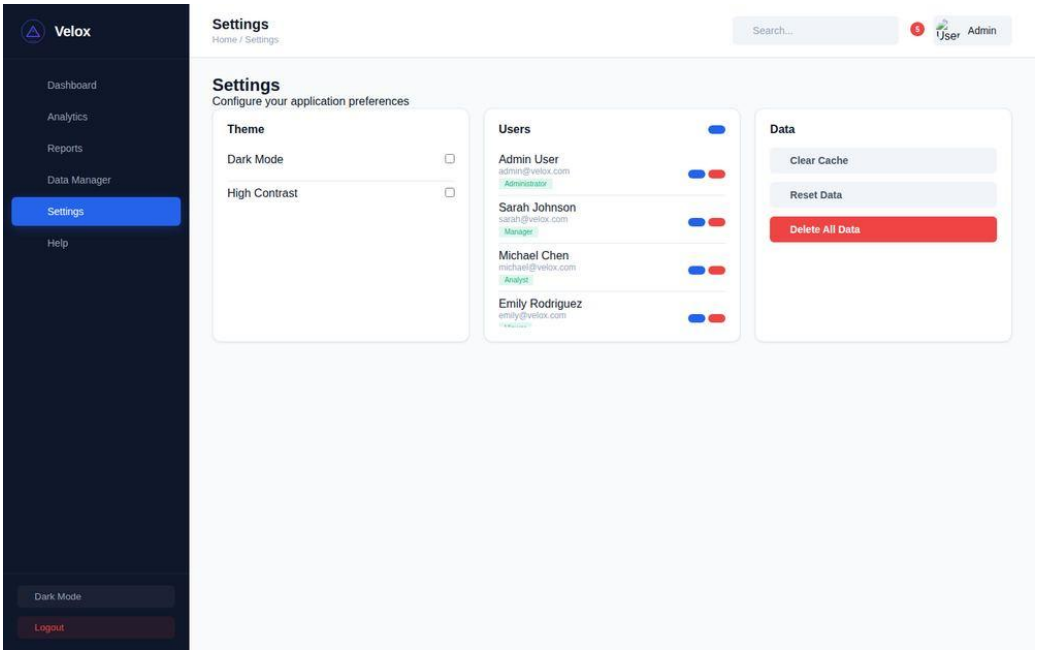


Figure 4.6: Settings — Theme, High Contrast and User Management.

Figure 4.7: Velox Analytics Dashboard — Mobile Responsive View.

4.28. GitHub Repository

GitHub Repository: https://github.com/mahnoor-yasir/Progree_Internship/tree/main/Task%204%20Enterprise%20Analytical%20Administration%20Portal

4.29. LinkedIn

LinkedIn Post: https://lnkd.in/p/dxdAmfCv

4.30. Learning Outcomes

- Centralising derived KPI figures in dataService.js rather than letting each view compute its own numbers was the difference between a dashboard that could silently drift out of sync after an edit and one that could not.
- Chart.js instances do not automatically pick up new CSS custom property values, so a theme toggle has to actively re-create chart instances rather than simply relying on the stylesheet swap.
- Composing filter predicates at the calling view, rather than hard-coding a fixed combination inside the filter engine, made it straightforward to support arbitrary combinations of search, category, status and region without a dedicated combined-filter function.
- Using discrete, single-purpose localStorage keys per preference (theme, contrast, font size, saved filters) avoided the risk of one write overwriting unrelated state in a single shared blob.

4.31. Future Enhancements

- A real backend API to replace the mock JSON dataset and localStorage persistence.
- An automated cross-browser and responsive test suite to replace the current manual verification.
- Formal accessibility auditing against WCAG 2.1 to move from “not independently verified” to a documented compliance level.
- Server-side authentication and role-based access control in place of the current mock login.

4.32. Result

Every official Task 4 requirement is implemented and traceable to a specific module, as shown in Section 4.19. Beyond the minimum scope, the delivered project also includes mock authentication with a role concept, full CRUD with localStorage persistence, pagination, quick filters, saved filters, a notification centre, PDF/CSV export, a high-contrast mode and font-size control, and a Help Center with FAQs and a keyboard-shortcut reference.

5. Overall Conclusion

The four tasks of the Progree Frontend Development Internship covered a progressively broader slice of frontend engineering. Task 1 established professional communication by announcing the internship publicly on LinkedIn alongside the official offer letter. Task 2 (LaunchFlow) applied semantic HTML5 and hand-written responsive CSS to a marketing landing page, verified against six breakpoint layers with no horizontal overflow. Task 3 (AERIS) built a REST API integration using the Fetch API and async/await, with typed error handling and dynamic DOM rendering driven entirely by live OpenWeatherMap data. Task 4 (Velox Analytics) combined client-side routing, Chart.js-based reactive data visualisation, multi-column searching, filtering and sorting, full CRUD operations, and a persisted dark-mode theme engine into a single enterprise analytics portal.

Across all four tasks, the consistent thread is frontend code written without frameworks or a backend: semantic markup, hand-written CSS layout systems, and vanilla JavaScript state management backed by localStorage. Each submission documents its own testing honestly, distinguishing verified behaviour from behaviour that was not independently re-tested for this report.

5.1. Skills Demonstrated by Task

Task	Core Skills Demonstrated
Task 1 — LinkedIn Announcement	Professional communication; public presentation of technical work
Task 2 — LaunchFlow	Semantic HTML5; custom CSS with Grid/Flexbox; multi-breakpoint responsive design; client-side form validation
Task 3 — AERIS	REST API integration; async/await and Promise composition; typed error handling; defensive UI rendering
Task 4 — Velox Analytics	Client-side routing; Chart.js data visualisation; multi-column search/filter/sort; CRUD architecture; state persistence

6. References

- MDN Web Docs — https://developer.mozilla.org/
- MDN — Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN — Window: localStorage — https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- MDN — CSS Filter Effects (backdrop-filter) — https://developer.mozilla.org/en-US/docs/Web/CSS/filter
- Chart.js Documentation — https://www.chartjs.org/docs/latest/
- jsPDF Documentation — https://github.com/parallax/jsPDF
- OpenWeatherMap API Documentation — https://openweathermap.org/api
- Font Awesome Documentation — https://fontawesome.com/docs

7. Project Links

Version control, source-code history and the canonical copy of all four submissions live in a single GitHub repository, organised into per-task folders:

Master GitHub Repository: https://github.com/mahnoor-yasir/Progree_Internship

Task	GitHub Repository	LinkedIn
Task 1 — LinkedIn Announcement	Repository Link	LinkedIn Post
Task 2 — LaunchFlow	Repository Link	LinkedIn Post
Task 3 — AERIS	Repository Link	LinkedIn Post
Task 4 — Velox Analytics	Repository Link	LinkedIn Post